# End-to-End Tracing for Apicurio + Kafka + Quarkus (Avro) - Local, No Docker

# End-to-End Tracing for Apicurio + Kafka + Quarkus (Avro)

### Local setup, no Docker - distributed tracing, wire-format inspection, and timing

---

## Overview

This guide covers three layers needed for full visibility into a local Apicurio Registry + Kafka + Quarkus (event-producer / event-consumer) Avro pipeline:

1. **Distributed tracing** - the shape and duration of the end-to-end flow
2. **Wire-format inspection** - the actual Avro bytes and schema IDs on the wire
3. **Time on wire** - precise latency between produce and consume

Everything below runs as native binaries or JARs - **no Docker required**.

---

## 1. Distributed Tracing

### Option A - Logging exporter (zero extra infrastructure)

Quarkus's built-in OpenTelemetry logging exporter prints full trace/span data straight to the console or log file. This is the fastest way to see the end-to-end flow with no extra process running.

```
quarkus.otel.traces.exporter=logging
quarkus.otel.simple=true
```

You will see log lines such as:

```
traceId=4bf92f3577b34da6a3ce929d0e0e4736, spanId=...,
parentSpanId=..., \
  name=produce, duration=12.4ms
```

Grep by `traceId` across both application logs to reconstruct one complete flow spanning producer -> broker -> consumer.

### Option B - Real UI, still no Docker

Jaeger ships as a single native binary:

```
        curl -L
```

```
https://github.com/jaegertracing/jaeger/releases/download/v1.6x.x/jaeger-
2.x.x-linux-amd64.tar.gz | tar xz
        ./jaeger-all-in-one
```

Point Quarkus at the OTLP endpoint:

```
quarkus.otel.exporter.otlp.endpoint=http://localhost:4317
quarkus.otel.traces.enabled=true
```

Open the UI at `http://localhost:16686`. Kafka client instrumentation auto-propagates trace context via record headers, so a single trace spans the produce call, the broker ack, and the consume/process step - each shown with real durations.

---

# 2. Wire-Format / Byte-Level Inspection

Apicurio's Avro wire format is:

```
[magic byte 0x0][4-byte schema ID][avro binary payload]
```

If using the **headers strategy** (`apicurio.registry.headers.enabled=true`), the schema ID sits in a Kafka header (`apicurio.value.globalId`) instead of the payload - easier to inspect directly.

## kcat (native install, no Docker)

```
        brew install kcat         # macOS
        sudo apt install kafkacat # Debian/Ubuntu
```

```
        kcat -C -b localhost:9092 -t your-topic \
          -f 'Headers: %h\nKey: %k\nPayload len: %S\n' -c 5
```

## AKHQ (runnable JAR, no Docker)

```
        curl -L -o akhq.jar
https://github.com/tchiotludo/akhq/releases/latest/download/akhq.jar
        java -jar akhq.jar
```

`application.yml` (place next to `akhq.jar`):

```
        akhq:
          connections:
            local-kafka:
              properties:
                bootstrap.servers: "localhost:9092"
              schema-registry:
                type: "apicurio"
                url: "http://localhost:8080/apis/registry/v2"   # adjust
port/version

          server:
            port: 8085   # avoid clashing with Quarkus apps on 8080/8081
```

Run it:

```
        java -Dmicronaut.config.files=application.yml -jar akhq.jar
```

Open `http://localhost:8085`. The Data tab decodes Avro payloads live, resolving schemas via Apicurio using the global ID found in the payload or headers.

> **Check first:** confirm the actual port Apicurio Registry is running on, and whether you're using the v2 (`/apis/registry/v2`) or v3 (`/apis/registry/v3`) API path - mismatches here are the most common cause of "schema not found" errors in AKHQ.

### Raw byte dump via bundled Kafka CLI

No extra tooling needed - use the Kafka CLI already bundled with a local Kafka install:

```
        kafka-console-consumer.sh --topic your-topic --bootstrap-server
localhost:9092 \
          --property print.headers=true --property print.timestamp=true
```

### TRACE-level logging (Apicurio + Kafka client internals)

```
quarkus.log.category."io.apicurio.registry".level=TRACE
quarkus.log.category."org.apache.kafka.clients".level=DEBUG
```

This surfaces schema lookups, cache hits/misses, and the resolved global ID for every record.

---

## 3. Time on Wire

Kafka provides two timestamps natively:

- `CreateTime` - set by the producer at send time
- `LogAppendTime` - set by the broker on append (requires topic config `message.timestamp.type=LogAppendTime`)

```
        kafka-console-consumer.sh --topic your-topic --bootstrap-server
localhost:9092 \
          --property print.timestamp=true
```

For precise, per-message wire latency independent of these, add a custom timestamp header at produce time and diff it against arrival time at consume - see the interceptor code below.

For aggregate metrics, Kafka exposes JMX metrics such as `record-queue-time-avg`, `request-latency-avg` (producer) and `fetch-latency-avg`, `records-lag` (consumer). Quarkus apps with `quarkus-micrometer-registry-prometheus` already expose these; scrape with a locally-installed Prometheus binary (also a single binary, no Docker needed).

---

## 4. Wire-Latency Kafka Interceptors (Code)

These operate at the raw `ProducerRecord` / `ConsumerRecord` level, so they work regardless of the Avro/Apicurio serialization layer.

### Producer interceptor (event-producer app)

```
        package com.example.interceptor;

        import org.apache.kafka.clients.producer.ProducerInterceptor;
        import org.apache.kafka.clients.producer.RecordMetadata;
        import org.apache.kafka.clients.producer.ProducerRecord;
        import org.apache.kafka.common.header.internals.RecordHeader;

        import java.nio.ByteBuffer;
```

```
        import java.util.Map;

        public class WireTimingProducerInterceptor implements
ProducerInterceptor<Object, Object> {

            @Override
            public ProducerRecord<Object, Object>
onSend(ProducerRecord<Object, Object> record) {
                long nowNanos = System.nanoTime();
                long nowEpochMillis = System.currentTimeMillis();

                ByteBuffer buf = ByteBuffer.allocate(16);
                buf.putLong(nowNanos);
                buf.putLong(nowEpochMillis);

                record.headers().add(new RecordHeader("x-wire-send-ts",
buf.array()));
                return record;
            }

            @Override
            public void onAcknowledgement(RecordMetadata metadata, Exception
exception) {
                // optional: log broker ack latency here if needed
            }

            @Override
            public void close() {}

            @Override
            public void configure(Map<String, ?> configs) {}
        }
```

## Consumer interceptor (event-consumer app)

```
        package com.example.interceptor;

        import org.apache.kafka.clients.consumer.ConsumerInterceptor;
        import org.apache.kafka.clients.consumer.ConsumerRecords;
        import org.apache.kafka.clients.consumer.ConsumerRecord;
        import org.apache.kafka.common.header.Header;
        import org.jboss.logging.Logger;

        import java.nio.ByteBuffer;
        import java.util.Map;

        public class WireTimingConsumerInterceptor implements
ConsumerInterceptor<Object, Object> {

            private static final Logger LOG =
Logger.getLogger(WireTimingConsumerInterceptor.class);

            @Override
            public ConsumerRecords<Object, Object>
onConsume(ConsumerRecords<Object, Object> records) {
                long consumeNanos = System.nanoTime();

                for (ConsumerRecord<Object, Object> record : records) {
                    Header header = record.headers().lastHeader("x-wire-
send-ts");
                    if (header != null) {
                        ByteBuffer buf = ByteBuffer.wrap(header.value());
                        long sentNanos = buf.getLong();
                        long sentEpochMillis = buf.getLong();
```

```
                        long wireLatencyNanos = consumeNanos - sentNanos;
                        double wireLatencyMs = wireLatencyNanos /
1_000_000.0;

                        LOG.infof("[WIRE-TIMING] topic=%s partition=%d
offset=%d sentEpochMs=%d wireLatencyMs=%.3f",
                                record.topic(), record.partition(),
record.offset(),
                                sentEpochMillis, wireLatencyMs);
                    }
                }
                return records;
            }

            @Override
            public void onCommit(Map<org.apache.kafka.common.TopicPartition,
org.apache.kafka.clients.consumer.OffsetAndMetadata> offsets) {}

            @Override
            public void close() {}

            @Override
            public void configure(Map<String, ?> configs) {}
        }
```

### Wiring (application.properties)

```
# event-producer
mp.messaging.outgoing.your-
channel.interceptor.classes=com.example.interceptor.WireTimingProduce


# event-consumer
mp.messaging.incoming.your-
channel.interceptor.classes=com.example.interceptor.WireTimingConsume
```

> **Important clock-skew note:** `System.nanoTime()` is only valid for diffing within the *same JVM*. Since the producer and consumer run in separate JVMs, `wireLatencyNanos` in the code above is not meaningful across processes - that's why `sentEpochMillis` is also captured. Use `consumeEpochMillis - sentEpochMillis` for a true cross-process wall-clock latency (accurate as long as both machines are NTP-synced, which is the default on localhost).

---

## Putting It All Together

| Layer | Tool | What it shows |
|---|---|---|
| Trace shape / flow | OTel logging exporter or Jaeger binary | Span-level produce -> broker -> consume timeline |
| Byte / schema inspection | kcat, AKHQ, Kafka CLI, TRACE logs | Raw Avro bytes, schema IDs, header values |
| Precise wire latency | Custom timestamp header + interceptors | True produce -> consume latency |
| Aggregate latency trends | JMX + Micrometer + Prometheus | `record-queue-time-avg`, `fetch-latency-avg`, lag |

A practical workflow: use the OTel logging exporter (or Jaeger) to see the overall flow shape, the custom epoch-millis header to get precise wire latency per message, and AKHQ or `kcat` to inspect bytes and schema IDs whenever something looks off.